

AWS Learning Notes

IAM Fundamentals, Hands-on Labs, EBS Volumes and CLI Roadmap

1. Root User vs IAM User

Logging in as the root user and as an IAM user shows the **same resources**, because both belong to the same AWS account. The difference is not what you can see but what you are *allowed to do*.

- An IAM user with an admin-type policy (AdministratorAccess) can do almost everything root can in services like EC2 and S3.
- **Only root** can do certain account-level tasks: close the account, change the account's root email and password, and manage some billing settings (unless billing access is enabled for IAM).
- Best practice: enable MFA on root, then use an IAM user or role for daily work.

2. EBS Volumes (EC2 Storage)

EBS (Elastic Block Store) provides disk-like storage for EC2 instances.

- The volume created with an instance is its **boot (root) volume**, typically 8 GiB. It has a Snapshot ID because it was created from the AMI.
- A volume you create manually has no snapshot and is **not attached** to anything until you attach it.
- A volume must be in the **same Availability Zone** as the instance it attaches to.

Attach and use an extra volume

- 1 EC2 > Instances: note the instance's Availability Zone.
- 2 EC2 > Volumes: select the volume, choose Actions > Attach volume, pick the instance.
- 3 SSH into the instance, check the disk name with `lsblk` (often `nvme1n1`), then format and mount it.

```
lsblk
sudo mkfs -t ext4 /dev/nvme1n1
sudo mkdir /data
sudo mount /dev/nvme1n1 /data
```

Create a file in `/data`, then try Actions > Create snapshot to see how EBS backups work.

Cost cleanup: unattached volumes and snapshots still cost a little. When finished, detach and delete the volume and delete the snapshot. Terminating an instance deletes its boot volume (by default).

3. IAM: The Mental Model

IAM (Identity and Access Management) answers two questions:

- **Authentication:** who are you? (a login or access key)
- **Authorization:** what are you allowed to do? (policies)

Principals (who can make requests)

Principal	Description
Root user	Account owner with unlimited power. Use rarely.
IAM user	Long-term identity for one person or application.
IAM role	Temporary identity assumed by a user, an AWS service (e.g. EC2) or another account. No password or permanent keys, so it is safer.

How AWS decides (policy evaluation)

- 1 Everything is **denied by default** (implicit deny).
- 2 An **Allow** in an attached policy opens that action.
- 3 An **explicit Deny always wins** over any Allow.

A brand-new IAM user can do nothing until a policy is attached. When a user has several policies (directly attached or through groups), AWS combines all the Allows.

Key IAM terms

Term	Meaning
User	A person or app with long-term credentials
Group	A collection of users sharing permissions
Role	Temporary permissions assumed by users or services
Policy	A JSON document that allows or denies actions
Action	An API operation written as service:Action, e.g. s3:CreateBucket

4. Lab 1: A Restricted User

Goal: see IAM block an action.

- 1 **Check the current user:** IAM > Users > (your user) > Permissions. Note the attached policy (likely AdministratorAccess = Allow everything).
- 2 **Create the user:** IAM > Users > Create user. Name it test-reader. Enable console access with a custom password.
- 3 **Attach a policy:** choose Attach policies directly and select AmazonS3ReadOnlyAccess. Create the user and copy the console sign-in URL.
- 4 **Test:** sign in as test-reader in a different browser (or after signing out). Listing S3 buckets works. Creating a bucket or an EBS volume is denied.

Reading the error

To create a bucket, the `s3:CreateBucket` permission is required.

- **s3:CreateBucket** is the exact action that was blocked, which is the name you write in policies.
- `AmazonS3ReadOnlyAccess` allows only read and list actions (`s3:Get*`, `s3:List*`). Nothing allows `CreateBucket`, so the default rule applies: **implicit deny**.

5. Lab 2: Groups

Attaching policies user by user gets messy. A **group** holds permissions, and anyone added to it receives them, so permissions are managed in one place.

- 1 **Create the group:** IAM > User groups > Create group. Name it s3-writers and attach AmazonS3FullAccess.
- 2 **Add the user:** open the group > Users tab > Add users > select test-reader.
- 3 **Test:** as test-reader, create an S3 bucket with a globally unique name. It should now succeed. Delete the bucket afterwards.
- 4 **Remove access:** remove test-reader from the group and try again. It is denied again.

Lesson: test-reader keeps its directly attached read-only policy and gains full access through the group. AWS takes the **union** of all Allows. Removing the group membership removes the extra access.

6. Do I Need the AWS CLI?

Not right now. Core services can be learned entirely in the console, which is often better for understanding concepts. The CLI becomes useful for jobs, automation, speed and repeatable scripts.

Easiest way to start: CloudShell

CloudShell is the terminal button in the console footer. It is free, runs in the browser, is already logged in as your current user, and has the CLI pre-installed. Try:

```
aws sts get-caller-identity
aws s3 ls
```

Installing the CLI on your own computer is only needed later, and that is when access keys come in, which are the risky part.

7. What Is Still To Learn

IAM

Level	Topics
Core	Policy JSON (Effect, Action, Resource, Condition, wildcards, explicit Deny); roles and trust policies; MFA and access keys
Intermediate	Resource-based policies (S3 bucket policies); conditions (IP, MFA, tags); STS and AssumeRole; credential report, last-accessed data, Access Analyzer, Policy Simulator, CloudTrail; password policy
Awareness	Permissions boundaries; AWS Organizations and SCPs; IAM Identity Center (SSO)

CLI and SDK

- **CLI:** aws configure, named profiles, --query and --output, aws s3 vs aws s3api, assuming roles, CloudShell.
- **SDK (boto3, Python):** clients vs resources, the credential chain (env vars, profiles, instance role), S3 upload/download, pagination, handling ClientError (AccessDenied).

Suggested order: policy JSON, roles, CloudShell and CLI basics, then boto3 only when needed.

8. Safety Habits

- Enable MFA on root and do not use root for daily work.
- **Never put access keys in code or commit them to GitHub.** Bots scan public repos and abuse leaked keys within minutes.
- Grant least privilege: only the permissions actually needed.
- Delete test users, buckets, volumes and snapshots when finished, and keep a billing budget alert.